

Yes — this is **Week 4**, covering:

- **Session: Model evaluation & comparison**
- **Practical: SVMs and model selection / meta-parameter tuning**
- **Session: Traditional Time Series + A/B Testing**

Below are your **exam-focused Week 4 notes** in the same two formats.

Week 4 — Model Evaluation and Comparison

Plus Time Series and A/B Testing

Part A — Structured & Formal Summary

1. Purpose of Week 4

Week 4 focuses on two separate but important topics:

1. **Correct model evaluation when comparing many models or tuning meta-parameters**
2. **Traditional time series analysis and A/B testing**

The first part is about avoiding **procedural overfitting**. This happens when the test set is used not only for final evaluation, but also to make choices about which model or hyperparameters to use.

The second part introduces traditional forecasting logic, especially how time series can be decomposed into trend, seasonality, cycles and remainder, then modelled using AR, MA and ARIMA-style approaches. It also introduces A/B testing as a real-world way to evaluate interventions.

Session — Model Evaluation and Comparison

2. Predictive Machine Learning Revisited

The lecture revisits the core goal of predictive machine learning:

The task is to predict a target feature for unseen instances.

An unseen instance means the full conceptual object has not been seen before. Both its input and output features are unknown until the model has already been finalised.

Examples from the lecture:

Instance	Output	Input features	Meaning of unseen
Bottle of wine	quality	physicochemical measurements	unopened wine not tasted by experts
Person	gender	store-visit counts	person not previously considered
Tweet	sentiment	tweet content	future tweet not manually labelled

This matters because evaluation must simulate what happens in reality: after the model is deployed, you cannot go back and change it using the new data.

3. Evaluation as a Simulation of Reality

The lecture explains that machine learning evaluation works by **simulating reality**.

In reality:

- before deployment, you can change the model,
- after deployment, new cases arrive,
- you cannot change the model based on those future cases before predicting them.

A hold-out test set simulates those future unseen cases. The test set represents the data you would not have known about if the project had started earlier.

The rule is:

After you evaluate on data beyond the “cannot change things” point, you may report the result, but you must not use that result to choose or change the model.

If you break this rule, your evaluation no longer reflects reality and you usually overestimate model performance.

4. Hold-Out Testing

A simple hold-out evaluation is:

1. Split the data into training and test sets.
2. Train the model on the training set.
3. Evaluate once on the test set.
4. Report the test score.

This gives an unbiased estimate of performance on new instances **only if the test set is not used to make model-selection decisions**.

Problem:

A randomly selected test set may be unusually easy or unusually hard. To reduce this randomness, we can use:

- repeated random resampling,
- cross-validation.

But cross-validation alone does not solve the problem if we use it incorrectly for both model selection and final performance reporting.

5. The Problem: We Rarely Fit Only One Model

In practice, we usually try many models:

- different algorithms,
- different feature sets,
- different preprocessing choices,
- different hyperparameters,
- different thresholds,
- different kernels,
- different regularisation values.

The lecture states that when we keep the same simple evaluation strategy while trying multiple models, we risk **procedural overfitting**.

6. Meta-parameters as Different Models

A key fact from the lecture:

Each combination of meta-parameters / hyperparameters should be considered a different model.

Why?

Because changing a meta-parameter changes model complexity.

Example:

For a decision tree regressor, `max_depth` controls the number of leaves and therefore the complexity of the fitted function. A shallow tree and a deep tree are not just "the same model with settings changed"; they are different candidate models with different flexibility.

Examples of model candidates:

- `RandomForestClassifier(max_depth=100)`
- `RandomForestClassifier(max_depth=10)`
- `SVC(C=1.0, kernel='rbf')`
- `SVC(C=10.0, kernel='rbf')`

- `SVC(C=1.0, kernel='linear')`
- `SVC(C=10.0, kernel='linear')`

Each should be treated as a separate candidate in model selection.

7. Normal Overfitting versus Procedural Overfitting

Normal overfitting

Normal overfitting means:

The model learns noise in the training set.

This is the usual overfitting problem from Weeks 1–3.

Procedural overfitting

Procedural overfitting means:

The model-selection procedure learns noise in the test set.

This happens when the test set is repeatedly reused to choose between models, tune hyperparameters, or adjust thresholds.

The lecture explains this using decision tree models of increasing complexity. The model with lowest test error may not be the best model for future data; it may simply match accidental noise shared between the training and test samples.

8. Why Procedural Overfitting Is Dangerous

When you choose a model based on repeated test-set performance, the test set stops being an unbiased estimate of future performance.

You have effectively used the test set as a validation set.

Consequences:

- the selected model may not generalise,
- reported test performance is too optimistic,
- the model may fail in deployment,
- business decisions may be based on exaggerated accuracy.

The lecture's key warning is that after evaluating on the "cannot change things" data, you can only report, not select.

Correct Evaluation Strategies

9. Strategy 1 — Basic Single Split

This is the simplest strategy when doing model selection.

Data is split into:

- training set,
- validation set,
- test set.

Process:

1. Split data into training/test.
2. Further split training into sub-training/validation.
3. Train each model candidate on sub-training data.
4. Score each candidate on validation data.
5. Select the best model using validation score.
6. Retrain the selected model on the full training set.
7. Evaluate once on the test set.
8. For deployment, retrain using all available data with the selected hyperparameters.

The test set is used only once, after model selection is complete.

Advantage

Simple and computationally cheap.

Disadvantage

Validation and test results may depend heavily on one lucky or unlucky split.

10. Why Retrain After Selecting Hyperparameters?

Once model complexity is fixed, adding more randomly sampled training data is unlikely to cause overfitting in the same procedural sense. More data usually helps performance.

Therefore, after selecting hyperparameters on validation data, you retrain the chosen model on the full training set before testing. For final deployment, you may retrain on all data with the chosen hyperparameters.

11. Strategy 2 — Advanced Single Split with Cross-Validation

This improves model selection stability.

Data is split into:

- training set,
- test set.

Then, inside the training set:

- cross-validation is used to compare model candidates.

Process:

1. Hold out a final test set.
2. Define a cross-validation strategy on the training set.
3. Evaluate each model/hyperparameter combination using the same CV folds.
4. Select the best model based on mean CV performance.
5. Retrain the best model on the full training set.
6. Evaluate once on the held-out test set.
7. Retrain on all data for deployment.

This ensures that per-model accuracies are not overly dependent on a single validation split.

Advantage

More reliable model selection than one validation split.

Disadvantage

The final test estimate still depends on one hold-out test split.

12. Strategy 3 — Nested Cross-Validation

Nested cross-validation is the most careful approach when you want both:

- stable model selection,
- unbiased performance estimation.

It uses:

- **inner cross-validation** for model selection,
- **outer cross-validation** for performance estimation.

Process:

1. Outer CV splits data into outer training and outer test folds.
2. For each outer training fold:
 - run inner CV to select hyperparameters,
 - train the selected model,
 - evaluate on the outer test fold.
3. Average outer test scores to estimate generalisation performance.
4. Check stability of selected hyperparameters across folds.
5. For deployment, rerun the full model-selection procedure on all data and train the final model.

The lecture says the selected parameters should ideally be stable. If they are unstable, you should measure/report this and consider whether the model should be used.

13. Why Not Always Use Nested Cross-Validation?

Nested CV is ideal conceptually, but not always practical.

Problems:

- computationally expensive,
- more complex to implement,
- many human decisions remain,
- if the dataset is very large, a representative hold-out test set may be enough,
- repeated inner/outer folds may waste time if the same hyperparameters are chosen every time.

The lecture notes that with enough data, a representative hold-out is often enough. With medium-sized data, a large enough test set plus advanced single split may be a practical compromise.

14. GridSearchCV

`GridSearchCV` automates model selection over a predefined grid of hyperparameters.

Example from the lecture:

- `max_depth = [5, 10, 100]`
- `max_features = ['sqrt', 'log2']`

This creates 6 Random Forest candidate models.

`GridSearchCV` :

1. Tries every parameter combination.
2. Evaluates each combination using cross-validation.
3. Selects the best combination.
4. Refits the best estimator on the full training data if `refit=True`.

The lecture shows that `GridSearchCV` is an easier way to implement the advanced single split approach for one model class.

15. Practical — SVMs and Model Selection

The practical uses a census salary prediction task:

Predict whether a person earns more than \$50K per year from census features.

The practical includes categorical input features, so it uses a pipeline with:

- `OneHotEncoder(handle_unknown='ignore') ,`
- `StandardScaler() ,`
- a classifier such as Logistic Regression, Random Forest or SVM.

The output feature is salary, converted into a binary classification target. The practical then asks students to evaluate multiple models and hyperparameters using increasingly careful evaluation methods:

Part A — Basic single split

- train/validation/test structure,
- compare candidate models,
- choose best from validation,
- evaluate once on test.

Part B — Advanced single split

- hold out test set,
- use cross-validation inside training data,
- compare candidates using mean CV score,
- evaluate selected model once on test.

Part B with GridSearchCV

- define a parameter grid,
- use `GridSearchCV` for inner model selection,
- evaluate final selected model on the held-out test set.

Part C — Nested CV

- use outer CV for unbiased evaluation,
- use inner CV / GridSearchCV for hyperparameter selection,
- train deployment version by rerunning model selection on all data.

Main practical message:

The more model choices you make, the more careful your evaluation procedure must be.

Session — Traditional Time Series + A/B Testing

Time Series

16. Univariate Parametric Prediction

A univariate time series uses a single measured variable over time.

Traditional time series prediction often works by assuming a simple parametric structure, then modelling that structure.

The lecture explains that prediction is achieved by finding structure in the data. One way to uncover structure is to assume a type of structure and then model it.

Traditional time series models often require low complexity because historically they were used with limited data. Too many parameters with too little data leads to overfitting.

17. Component-Based Structure

A time series can often be decomposed into components:

1. **Trend**
2. **Seasonal component**
3. **Cyclic component**
4. **Remainder / error**

The aim is to separate simple, meaningful patterns so that the remaining series is easier to model.

18. Trend Component

The trend is the long-term increase or decrease in the data.

It may be nonlinear.

Examples:

- long-term sales growth,
- long-term decline in demand,
- gradual increase in website traffic,
- long-term inflation effect.

Trend captures the broad direction of the series over time.

19. Seasonal Component

The seasonal component is a repeating pattern with a fixed period.

Examples:

- higher retail sales every December,
- weekly website traffic cycles,
- daily electricity demand patterns,
- monthly billing cycles.

The key property is that the period is fixed and known or discoverable.

20. Cyclic Component

The cyclic component is a slowly repeating pattern with no fixed period.

Examples:

- business cycles,
- economic cycles,
- multi-year demand cycles.

The lecture notes that trend and cycle components are usually considered jointly in decomposition.

21. Remainder Component

The remainder is what is left after trend, seasonality and cycle have been removed.

Ideally, the remainder should be "simple" and have nice statistical properties. The lecture later connects this to stationarity.

22. Additive and Multiplicative Decomposition

A component-based time series model can be:

Additive

Plain formula:

$$y_t = S_t + T_t + E_t$$

Meaning:

- observed value = seasonal component + trend component + error.

Use additive decomposition when seasonal variation stays roughly constant over time.

Multiplicative

Plain formula:

$$y_t = S_t * T_t * E_t$$

Meaning:

- observed value = seasonal component multiplied by trend and error.

Use multiplicative decomposition when seasonal variation increases as the level of the series increases.

The lecture notes that additive decomposition can still be used for multiplicative-looking data by applying a log transformation first.

23. Choosing Additive versus Multiplicative

Ask:

Does the magnitude of seasonal variation increase over time?

- If no, use additive.
- If yes, use multiplicative.

Example:

- constant seasonal swings around a trend: additive,
 - seasonal swings get larger as the series grows: multiplicative.
-

24. Why Decompose a Time Series?

The lecture gives two reasons:

24.1 Descriptive modelling

Remove uninteresting components and study the remainder.

24.2 Forecasting

Model and predict simpler components separately.

Trend and seasonality are often easier to forecast individually. The goal is to make the remainder easier to model.

25. Stationarity

A stationary time series is one whose statistical properties are constant over time.

These properties include:

- mean,
- variance,
- autocorrelation.

Traditional models such as ARIMA generally work better when the series or remainder is stationary.

Exam phrase:

Time series decomposition and differencing are often used to make the remaining series more stationary and therefore easier to forecast.

26. Classical Decomposition

Classical decomposition is an older intuitive method.

It estimates:

- trend-cycle using a moving average,
- seasonality using period averages.

For monthly data, this means:

- January values are replaced by January's average,
- February values by February's average,
- and so on.

Drawbacks

The lecture identifies important drawbacks:

- assumes seasonal component repeats from year to year,
 - moving average prevents decomposition across the full period because start/end points are missing,
 - now considered a weak or outdated approach compared with more advanced methods.
-

27. X-12-ARIMA

X-12-ARIMA is a more advanced decomposition protocol.

The lecture describes it as:

- used by many government agencies,
- designed for monthly and quarterly data,
- iteratively refining estimates of trend and seasonality,
- identifying and reducing outliers,
- handling dates with known effects such as holidays.

It is powerful but not generic; it focuses on specific types of seasonal adjustment.

28. STL Decomposition

STL stands for:

Seasonal and Trend decomposition using Loess.

The lecture says STL:

- handles any type of seasonality if you specify the period,
- allows trend-cycle smoothness to be controlled,

- can be robust to outliers,
- is usually robust to outliers if desired.

Drawbacks of STL

The lecture lists two main drawbacks:

1. No direct multiplicative version, although log transformation can be used as a counterargument.
2. It does not directly remove known predictors/effects, although the lecture questions whether we are always sure those effects are true predictors.

29. Detrending

Detrending means removing trend from a time series to make it easier to predict.

The lecture assumes seasonality has already been removed and says detrending is done by **differencing**.

Differencing changes the time series from values to changes in values.

Plain formula:

$$y'_t = y_t - y_{(t-1)}$$

Meaning:

- new value = current value minus previous value.

This is an invertible transformation and can be applied multiple times. The number of differencing operations can be treated as a parameter, often written as k .

AR, MA and ARIMA

30. Auto-Regressive Model — AR

An auto-regressive model predicts the current value using previous values of the same time series.

The lecture explains it by converting time points into regression-style rows.

Example:

- output: value at time t
- input: value at time $t-1$

This gives an AR1 model.

For AR2:

- output: value at time t

- input: value at time $t-2$, and generally earlier lags depending on the formulation.

The key idea is:

Past values become input features for predicting future/current values.

31. Moving Average Model — MA

The lecture warns that the MA model for forecasting is **not the same as moving average smoothing**.

A moving average forecasting model predicts values using previous forecast errors.

Conceptually:

- each value y_t is predicted by a linear combination of past forecast errors,
- past errors act like input features,
- each past error gets a learned weight.

The lecture notes that this model is poorly named because it is different from the smoothing moving average used earlier in decomposition.

32. ARIMA

ARIMA combines:

- AR terms,
- MA terms,
- differencing.

The lecture describes ARIMA conceptually as a linear regression model plus a preprocessing step.

ARIMA has:

AR terms

Use previous time-series values as predictors.

Parameter:

- p = number of historical values / autoregressive terms.

MA terms

Use previous forecast errors as predictors.

Parameter:

- q = number of historical errors.

I term

The "I" stands for integrated, which refers to differencing.

Parameter:

- k = number of times the data is differenced to make the process more stationary.

The lecture uses ARIMA(p, q, k), where p is AR terms, q is MA terms, and k is differencing.

33. ARIMA Extensions

The lecture notes that the basic ARIMA version has extensions, including:

- Seasonal ARIMA,
- ARIMA with external/domain variables such as week or working-day indicators.

This means time-series forecasting can include both temporal structure and additional explanatory variables.

A/B Testing

34. Definition of A/B Testing

A/B testing is the controlled statistical testing of an intervention in the real world.

It is used to determine whether an intervention has any positive or negative effect.

Examples:

- testing direct marketing campaigns,
 - testing website layouts,
 - testing sales funnel designs,
 - testing store layouts,
 - testing product updates.
-

35. A/B Testing Is Not Cross-Validation

The lecture explicitly states:

A/B tests are not cross-validation.

Why?

Cross-validation uses existing historical data. It cannot evaluate the causal effect of a new intervention in the live world.

A/B testing evaluates what happens when different users/stores/customers are exposed to different interventions.

36. Key Requirements for Valid A/B Testing

A valid A/B test must be carefully designed.

Important requirements from the lecture include:

36.1 Random assignment

Subjects must be randomly assigned to groups.

Without random assignment, differences may be due to pre-existing group differences rather than the intervention.

36.2 Decide exposure level

You must decide whether to run the test on:

- 100% of visitors,
- a smaller proportion,
- gradual rollout.

Large samples are useful, but risky designs should not be exposed to too many customers immediately. Gradual scale-up may reduce risk.

36.3 Consider stratified sampling

Different user groups may behave differently.

Example:

- new users,
- returning users,
- subscribed users.

Stratified sampling may be useful to ensure important groups are represented and can be analysed separately.

36.4 Decide how many variations to test

Traditional A/B testing compares one alternative against the original.

Testing many variations at once requires more care and more subjects.

37. Technical Pitfalls in A/B Testing

The lecture highlights several practical risks.

37.1 The intervention must work correctly

If the new website design accidentally changes search results or slows down the page, the test no longer isolates the design effect.

The measured effect may be caused by a bug or performance issue, not the intended intervention.

37.2 Users must receive consistent behaviour

If a user sees version A, closes the browser, returns later and sees version B, the test becomes invalid because the same user may appear in both groups.

This is especially important for e-commerce conversion tests.

37.3 Sample size must be large enough

A/B tests need enough subjects to support statistical testing.

A small sample may produce noisy, unreliable results.

38. Business Pitfalls in A/B Testing

A/B testing can harm the business if poorly planned.

The lecture lists several business concerns:

38.1 Intervention must be actionable

There is no point testing something the business cannot realistically adopt.

Example: testing an unsustainable in-store promotion may be a bad idea even if it increases sales.

38.2 Cost-benefit must be considered

Even if conversion increases, the test may not be worth:

- the cost of running it,
- the risk of customer loss,
- employee time,
- operational disruption.

38.3 Plan transition after the test

Users may build habits around the test version. Returning them to the old version may cause dissatisfaction.

The business must decide how users will transition back or move permanently to the new version.

38.4 QA and sign-off matter

The business must know:

- who signs off the intervention,
 - how QA handles the A/B test,
 - whether the test affects system performance,
 - whether it disrupts ordinary business operations.
-

Week 4 High-Yield Exam Checklist

You should be able to explain:

- why evaluation must simulate unseen future data,
 - why test data cannot be used for model selection,
 - normal overfitting versus procedural overfitting,
 - why each hyperparameter combination is a different model,
 - basic single split evaluation,
 - advanced single split with cross-validation,
 - nested cross-validation,
 - why GridSearchCV is useful,
 - why final test evaluation happens only after model selection,
 - why deployment models are retrained on all data,
 - time-series trend, seasonality, cyclic and remainder components,
 - additive versus multiplicative decomposition,
 - stationarity,
 - classical decomposition and its drawbacks,
 - STL and X-12-ARIMA,
 - differencing and detrending,
 - AR, MA and ARIMA intuition,
 - A/B testing definition,
 - why A/B testing is not cross-validation,
 - random assignment,
 - sample-size issues,
 - technical and business pitfalls of A/B tests.
-

Part B — Q&A Summary

1. What is the main goal of predictive machine learning?

To predict a target feature for unseen instances.

2. What does “unseen instance” mean?

It means the whole conceptual object has not been seen before. Both input and output features are unknown until after the model has been finalised.

3. Why is evaluation described as a simulation of reality?

Because evaluation tries to mimic what happens after deployment: new cases arrive, and the model must predict them without being changed using those cases.

4. What is the “cannot change things” rule?

Once data is treated as future/test data, it can only be used for final reporting. It must not be used to choose or modify the model.

5. What happens if the test set is used for model selection?

The evaluation no longer reflects reality and the reported performance becomes overly optimistic.

6. What is hold-out testing?

Hold-out testing splits data into training and test sets, trains on training data, and evaluates once on the test set.

7. When is hold-out testing unbiased?

When the test set is not used to make any model-selection decisions.

8. Why might one train-test split be unreliable?

Because the test set may be unusually easy or hard due to random sampling.

9. What can reduce dependence on a lucky or unlucky split?

Repeated resampling or cross-validation.

10. Why is cross-validation alone not always enough?

If cross-validation results are used to repeatedly choose models and also report final performance, procedural overfitting can still occur.

11. What is procedural overfitting?

Procedural overfitting occurs when the model-selection procedure overfits to the validation/test data used to choose models.

12. How is procedural overfitting different from normal overfitting?

Normal overfitting is fitting noise in the training data. Procedural overfitting is fitting noise in the test or validation data through repeated model selection.

13. Why is procedural overfitting dangerous?

It makes the selected model look better than it will be on truly new data.

14. What is a meta-parameter or hyperparameter?

A setting chosen before training that controls model complexity or behaviour, such as tree depth, C, gamma or kernel type.

15. Why is each hyperparameter combination treated as a different model?

Because each combination changes the model's complexity and behaviour.

16. Give an example of hyperparameter combinations as different models.

An SVM with `C=1, kernel='rbf'` and an SVM with `C=10, kernel='linear'` are different candidate models.

17. What is the basic single split strategy?

Split data into training, validation and test sets. Use validation to select the model, then use test once for final evaluation.

18. What is the weakness of basic single split?

The validation or test result may depend heavily on one random split.

19. What is the advanced single split strategy?

Hold out a test set, then use cross-validation within the training set to choose the model.

20. Why is advanced single split better than basic single split?

It makes model selection less dependent on one validation split.

21. What is nested cross-validation?

Nested CV uses inner CV for model selection and outer CV for unbiased performance estimation.

22. What is inner CV used for?

Choosing the best model or hyperparameters.

23. What is outer CV used for?

Estimating how the whole model-selection procedure performs on unseen data.

24. Why is nested CV useful?

It provides a less biased estimate when many models or hyperparameters are compared.

25. Why not always use nested CV?

It is computationally expensive and can be complex to set up.

26. When may a representative hold-out test set be enough?

When there is enough data for the hold-out set to reliably represent future data.

27. What is GridSearchCV?

A sklearn tool that tests all combinations in a parameter grid using cross-validation and selects the best one.

28. What does `refit=True` do in GridSearchCV?

It refits the best model on the full training data after selecting the best parameters.

29. Why should the same CV splits be used for each model?

To make model comparisons fair because each model is evaluated on the same training/validation partitions.

30. Why retrain the final model after selecting hyperparameters?

Because once model complexity is fixed, using more data can improve performance without the same model-selection overfitting risk.

31. What was the practical task?

Predict whether a person earns more than \$50K per year using census data.

32. Why did the practical need OneHotEncoder?

Because the census dataset contains categorical input features.

33. Why did the practical use `handle_unknown='ignore'` ?

So the encoder can handle categories in validation/test data that were not seen during fitting.

34. Why was StandardScaler included in the pipeline?

To scale encoded features before models such as logistic regression or SVM.

35. What is the main lesson from the model-selection practical?

The more models or hyperparameters you try, the more careful your validation and test strategy must be.

Time Series Q&A

36. What is a univariate time series?

A sequence of observations of one variable over time.

37. What does parametric prediction mean in time series?

It means assuming a model structure with parameters and fitting that structure to the time series.

38. Why do traditional time series models often prefer low complexity?

Because they were often designed for smaller datasets, and too many parameters can overfit.

39. What are the main time-series components?

Trend, seasonal component, cyclic component and remainder.

40. What is trend?

A long-term increase or decrease in the series.

41. What is seasonality?

A repeating pattern with a fixed period.

42. What is a cyclic component?

A repeating pattern with no fixed period, often slower and longer-term than seasonality.

43. What is the remainder?

The leftover part after trend, seasonality and cycle are removed.

44. Why do we decompose a time series?

To separate simpler components, understand structure, and make the remaining series easier to model.

45. What is additive decomposition?

A decomposition where the observed value is the sum of seasonal, trend and error components.

Plain formula:

$$y_t = S_t + T_t + E_t$$

46. What is multiplicative decomposition?

A decomposition where the observed value is the product of seasonal, trend and error components.

Plain formula:

$$y_t = S_t * T_t * E_t$$

47. When should additive decomposition be used?

When seasonal variation stays roughly constant over time.

48. When should multiplicative decomposition be used?

When seasonal variation increases as the overall level of the series increases.

49. How can multiplicative data sometimes be handled with additive methods?

By applying a log transformation first.

50. What is stationarity?

A time series is stationary when properties such as mean, variance and autocorrelation are constant over time.

51. Why is stationarity useful?

Traditional models such as ARIMA often work better on stationary data.

52. What is classical decomposition?

An older decomposition method that estimates trend-cycle using moving averages and seasonality using period averages.

53. What are drawbacks of classical decomposition?

It assumes seasonality repeats exactly and loses values at the start and end because of moving averages.

54. What is X-12-ARIMA?

A government-used decomposition protocol for monthly and quarterly data that refines trend and seasonality, handles outliers and known date effects.

55. What is STL?

Seasonal and Trend decomposition using Loess.

56. What are advantages of STL?

It handles different seasonal periods, allows control over smoothness, and can be robust to outliers.

57. What is detrending?

Removing trend from a time series to make it easier to model.

58. How is detrending done in this lecture?

By differencing.

59. What is differencing?

Subtracting the previous value from the current value.

Plain formula:

$$y'_t = y_t - y_{(t-1)}$$

60. What does differencing change?

It changes the series from values into changes in values.

61. Can differencing be applied more than once?

Yes. The number of differencing steps can be treated as a parameter.

62. What is an AR model?

An autoregressive model predicts the current value using previous values of the same time series.

63. What is AR1?

A model that uses the previous time point to predict the current time point.

64. What is an MA model in forecasting?

A model that predicts the current value using previous forecast errors.

65. Is MA forecasting the same as moving average smoothing?

No. The lecture explicitly says they are different despite the similar name.

66. What is ARIMA?

ARIMA combines autoregressive terms, moving-average error terms and differencing.

67. What does p mean in ARIMA?

The number of autoregressive lag terms.

68. What does q mean in ARIMA?

The number of previous forecast-error terms.

69. What does k mean in ARIMA according to the lecture?

The number of differencing steps used to make the series more stationary.

70. Why is ARIMA described as linear regression plus preprocessing?

Because AR and MA terms form weighted input features, while differencing acts as preprocessing.

A/B Testing Q&A

71. What is A/B testing?

A/B testing is controlled statistical testing of an intervention in the real world.

72. What is the purpose of A/B testing?

To determine whether an intervention has a positive, negative or no effect.

73. Give examples of A/B testing.

Testing marketing campaigns, website layouts, sales funnels, store layouts or product updates.

74. Why is A/B testing not cross-validation?

Cross-validation uses existing data. A/B testing evaluates the causal effect of a new live intervention.

75. Why must assignment be random in A/B testing?

To make groups comparable so differences can be attributed to the intervention rather than pre-existing differences.

76. Why might stratified sampling be useful?

Different user groups may behave differently, so stratification ensures important groups are represented.

77. Why not test many variations carelessly?

More variations require larger samples and more careful statistical design.

78. Why must users receive consistent behaviour?

If the same user sees both A and B, their outcome cannot be clearly assigned to one treatment.

79. Why is sample size important?

Small samples may not provide reliable statistical evidence.

80. Why must the intervention be technically correct?

If bugs or slow performance are introduced, the measured effect may not be due to the intended intervention.

81. Why must an A/B test be actionable?

There is no point testing a change the business cannot realistically adopt.

82. What business costs should be considered?

Customer loss, operational disruption, staff time, development cost, system performance and transition cost.

83. Why is transition planning important?

Users may form habits around the test version, so moving them back or forward must be planned.

84. What is the main exam message of Week 4?

When comparing models, protect the test set from model-selection decisions. For time series, understand components, decomposition, stationarity, differencing and ARIMA intuition. For A/B testing, remember that valid experiments require randomisation, consistency, sufficient sample size and business feasibility.